

PLANEJAMENTO FINAL DE ATIVIDADES - PROJETO GLICNUTRI

1. Identificação do projeto

- **Nome do projeto:** GlicNutri
 - **Entrega final:** 31 de maio de 2026
 - **Última revisão deste planejamento (Markdown/PDF):** 13 de maio de 2026 — consolidação de pacote final, checklists Bento/Thayse, LGPD, ZIP e ensaio (`entregas/PACOTE-MONTAGEM.md` , seção 11).
 - **Objetivo geral do planejamento:** organizar, de forma clara e acadêmica, as atividades necessárias para concluir o Projeto GlicNutri dentro do prazo, considerando as exigências das disciplinas do professor Bento e da professora Thayse. O planejamento contempla a consolidação do aplicativo e do banco de dados, com auditoria, logs e cuidados com LGPD, além da adequação da entrega de Machine Learning para utilizar dados reais do GlicNutri no Supabase, incluindo pipeline de ML, persistência de modelos e disponibilização por API.
-

2. Documentos e arquivos analisados

Foram considerados, para este planejamento, os seguintes documentos, notebooks e componentes do projeto:

- `Requisitos_Bento_GlicNutri.pdf`
 - `Requisitos_Thayse_GlicNutri.pdf`
 - `Diabetes_pipeline_ml.ipynb` (referência histórica no PDF)
 - `machine-learning/notebooks/referencia/glucobench-reference.ipynb` (GlucoBench)
 - `Cronograma ADS5 - Projeto de Sistemas V3.pdf`
 - Pastas e arquivos de aulas: AULA1 até AULA7 , utilizados como materiais de apoio para compreender os requisitos da disciplina da professora Thayse.
 - Código do aplicativo GlicNutri, desenvolvido em Expo/React Native e integrado ao Supabase.
 - Banco Supabase e migrations, contendo estrutura de dados, funções e componentes relacionados à persistência.
 - Checklist Bento (13 requisitos): `entregas/bento/checklist-13-requisitos-bento.md`
 - Pacote Bento (BD/CRUD/UX): `entregas/bento/BANCO-DE-DADOS-ER-EVIDENCIAS.md` , `entregas/bento/CRUD-VALIDACOES-ROTEIRO-EVIDENCIAS.md` , `entregas/bento/USABILIDADE-RELATORIOS-PRINTS-TEXTOS.md`
 - Resumo/Checklist Thayse (ML): `entregas/thayse/RESUMO-ENTREGA-ML.md`
 - Checklist pacote final: `entregas/PACOTE-FINAL-CHECKLIST.md`
 - Montagem ZIP + ensaio: `entregas/PACOTE-MONTAGEM.md` , `entregas/ENSAIO-FINAL-CHECKLIST.md` , `scripts/build-zip-entrega.ps1`
 - LGPD (texto curto): `entregas/LGPD-TEXTOS-CURTO.md`
 - Roteiro de slides: `entregas/slides-roteiro.md`
-

3. Diagnóstico geral do projeto

3.1 O que já existe no projeto

De forma geral, o projeto já possui uma base funcional consistente, incluindo:

- Aplicativo desenvolvido em Expo/React Native, com telas para perfis de usuário, como paciente, nutricionista e administrador.
- Integração com Supabase, utilizada para autenticação e persistência de dados.
- Fluxos de uso relevantes já implementados, como monitoramento e registro de glicemia, registro de medicação/insulina, registro de refeição com apoio de IA e painel administrativo com indicadores, auditoria e logs.
- Banco de dados versionado por migrations, o que facilita documentação, rastreabilidade e reprodução.
- Notebook de Machine Learning no repositório (`machine-learning/notebooks/glicnutri_ml_pipeline.ipynb`) com pipeline paciente-dia; usar CSV exportado do Supabase (`machine-learning/scripts/export_supabase_csv.py`) para dados reais ou o CSV de exemplo (`machine-learning/data/sample_glicnutri_patient_day.csv`) para desenvolvimento local.

3.2 O que está incompleto

Apesar da base do sistema existir, há pontos que precisam ser finalizados e padronizados para o contexto acadêmico:

- A entrega de Machine Learning precisa estar **organizada como evidência acadêmica** (prints, métricas e explicação), mas **já foi executada com dados reais exportados do Supabase** (CSV + manifest + treino + artefatos).
- Backend Python com FastAPI e endpoint `POST /predict` existe no repositório (`machine-learning/api/app/main.py`) e **já foi integrado ao app para teste local** (tela paciente "Previsão (IA)"). Hospedagem em nuvem permanece opcional para a apresentação final.
- As evidências e a documentação acadêmica foram **consolidadas no repositório** (checklists, roteiros, LGPD, pacote, ensaio); falta apenas a **entrega física** (.pptx, vídeo, ZIP gerado, Word final PDF) conforme `entregas/PACOTE-MONTAGEM.md`.
- Relatórios e gráficos de gestão existem parcialmente, mas devem ser organizados como entregável, com explicação e evidências.

3.3 O que ainda falta implementar

- (Concluído) Executar o export `machine-learning/scripts/export_supabase_csv.py` com credenciais reais e volume suficiente para treino (evidência: `machine-learning/data/export_manifest.json` + `glicnutri_patient_day_export.csv`).
- (Concluído) Apontar o notebook `machine-learning/notebooks/glicnutri_ml_pipeline.ipynb` para o CSV real e refinar pré-processamento (linhas com `glucose_mean_mg_dl` nulo são removidas do treino supervisionado).
- (Concluído) Treinar e avaliar os quatro modelos com dados reais (classificação, regressão, clusterização e similaridade) e persistir artefatos.
- (Concluído) Servir o modelo em ambiente de demonstração local (uvicorn) e registrar evidências de chamadas à API (inclui correção de CORS para app web).
- Consolidar documentação final, evidências, prints, roteiros de demonstração e checklist de entrega (**suporte concluído no repositório; ver secção 11 e `entregas/PACOTE-MONTAGEM.md`**).

3.4 Status real no repositório (sincronização técnica — maio/2026)

Esta subsecção amarra **requisito** → **evidência no código** conforme o estado atual do repositório (não substitui documentos PDF/Word externos nem vínculos acadêmicos que dependem de prints).

Área	Situação no repo	Onde evidenciar
App Expo + Supabase	Implementado: auth (paciente/nutri/admin/Google), cadastros, monitoramento, refeição IA, painel admin	GlicNutri/src/ , GlicNutri/supabase/migrations/
Export CSV reproduzível	Script Postgres paciente-dia + procedimento documentado + atalhos	machine-learning/scripts/export_supabase_csv.py (DATABASE_URL); machine-learning/EXPORT_CSV.md ; npm run ml:export-csv ; machine-learning/scripts/exportar_csv_semana2.ps1 ; manual entregas/SEMANA-2-PASSO-A-PASSO.md
Dataset / exemplo	CSV sintético para notebook sem DB	machine-learning/data/sample_glicnutri_patient_day.csv
Notebook ML (GlicNutri paciente-dia)	Pipeline (EDA + 4 tarefas + joblib)	machine-learning/notebooks/glicnutri_ml_pipeline.ipynb
Notebook ML (GlucoBench referência)	Carga/validação GlucoBench	machine-learning/notebooks/referencia/glocobench-reference.ipynb
Artefatos joblib	Gerados pelo notebook ou machine-learning/api/scripts/fit_demo_models.py	machine-learning/api/artifacts/*.joblib , training_meta.json
API POST /predict	FastAPI + contrato JSON	machine-learning/api/app/main.py , dependências em machine-learning/api/requirements.txt
Auditoria e logs (implementação no app)	Completo no código: gravação Storage, painel admin, recuperação de senha sem dados sensíveis nos detalhes	servicoAuditoria.js , TelaHomeAdmin.js , TelaAuditoriaAdmin.js , TelaLogsSistemaAdmin.js , TelaRecuperarSenha.js
Logs persistidos via Supabase Storage (bucket audit-logs)	Completo no código (fluxo implementado)	Idem + migrations/polices conforme projeto
Semana 2 — Bento / auditoria (evidências visuais em prints/)	COMPLETO (evidência validada no repositório)	evidencias-auditoria.md ; capturas em prints/
Bento — checklist 13 requisitos	Completo (repo)	entregas/bento/checklist-13-requisitos-bento.md
Bento — apoio para fechar “parciais” (BD/CRUD/UX/relatórios)	Completo (roteiros + texto)	entregas/bento/BANCO-DE-DADOS-ER-EVIDENCIAS.md , entregas/bento/CRUD-VALIDACOES-ROTEIRO-EVIDENCIAS.md , entregas/bento/USABILIDADE-RELATORIOS-PRINTS-TEXTO.md
Export CSV Supabase (dados reais)	COMPLETO (execução local)	CSV + manifest gerados: machine-learning/data/glicnutri_patient_day_export.csv , machine-learning/data/export_manifest.json (hash + contagem + colunas)
ML referência GlucoBench	Parcial / em andamento	machine-learning/notebooks/referencia/glocobench-reference.ipynb ; sem treino/pipeline final obrigatório nesta linha
Thayse — resumo/roteiro ML	Completo	entregas/thayse/RESUMO-ENTREGA-ML.md
Entregas acadêmicas (ZIP, slides, vídeo, Word único)	Suporte no repo; ficheiros finais pelo grupo	entregas/PACOTE-MONTAGEM.md , scripts/build-zip-entrega.ps1 , entregas/slides-roteiro.md , entregas/ENSAIO-FINAL-CHECKLIST.md

Nota sobre CRUD / exclusão: exclusão física de glicemia/medicação não é exposta por design; histórico usa ocultação no app + auditoria de eventos; exclusão de paciente pelo nutricionista é lógica (`excluido`).

Semana 2 (entrega prevista 10/05) — situação no repositório (atualizado maio/2026): **COMPLETA** no conjunto **Bento / auditoria e Thayse (export + manifest + registo em EXPORT_CSV.md + notebook executado + artefatos + API + integração na app)**. Evidência visual Semana 2: [evidencias-auditoria.md](#) e [prints/](#). Índice e manual: [entregas/README-ENTREGA-SEMANA-2.md](#) , [entregas/SEMANA-2-PASSO-A-PASSO.md](#) ; export: `npm run ml:export-`

csv e `machine-learning/scripts/exportar_csv_semana2.ps1` . Checklist de fluxos: `entregas/checklist-semana-2-fluxos-app.md` (sincronizado com artefatos).

3.4.1 Entregáveis no Git — Semana 2 (sincronização maio/2026)

Entregável	Estado no repositório	Onde
Índice da entrega Semana 2	Completo	<code>entregas/README-ENTREGA-SEMANA-2.md</code>
Manual do grupo (export, notebook, checklist)	Completo	<code>entregas/SEMANA-2-PASSO-A-PASSO.md</code>
Procedimento export + registo	Completo (registo preenchido)	<code>machine-learning/EXPORT_CSV.md</code>
Script export Python + atalhos npm/PowerShell	Completo	<code>machine-learning/scripts/export_supabase_csv.py</code> , <code>package.json</code> (<code>ml:export-csv</code>) , <code>machine-learning/scripts/exportar_csv_semana2.ps1</code>
Checklist Semana 2	Completo (R/T alinhados a manifest + notebook; B/C com notas)	<code>entregas/checklist-semana-2-fluxos-app.md</code>
Demo reunião 10/05	Roteiro no repo	<code>entregas/demo-reuniao-semana-2.md</code>
Bento — evidência visual Semana 2	Completo	<code>entregas/bento/semana-2-auditoria/</code>
Thayse — manifest / registo / notebook com dados reais	Completo (execução local)	<code>machine-learning/data/export_manifest.json</code> ; notebook executado: <code>machine-learning/notebooks/glicnutri_ml_pipeline.executed.ipynb</code>

3.5 Requisito → evidência → artefacto (resumo)

Foco	Estado	Onde comprovar
Bento — autenticação, fluxos, CRUD	Completo (repo + evidências Semana 2 + roteiros)	Código: <code>src/telas/autenticacao/</code> , <code>App.js</code> , <code>servicoDadosPaciente.js</code> ; pasta <code>entregas/bento/semana-2-auditoria/</code> — Evidência visual validada com capturas reais no repositório
Bento — auditoria (código)	Completo	<code>servicoAuditoria.js</code> , telas admin e fluxos em <code>checklist-auditoria.md</code>
Bento — auditoria (evidência visual Git, Semana 2)	Completo	<code>evidencias-auditoria.md</code> + <code>prints/</code>
Thayse — CSV reprodutível	Completo (manifest + registo + script)	<code>machine-learning/scripts/export_supabase_csv.py</code> , <code>machine-learning/data/export_manifest.json</code> , <code>machine-learning/EXPORT_CSV.md</code>
Thayse — ML GlicNutri (paciente-dia)	Completo (execução local)	<code>machine-learning/notebooks/glicnutri_ml_pipeline.ipynb</code> + execução <code>machine-learning/notebooks/glicnutri_ml_pipeline.executed.ipynb</code> + artefatos em <code>machine-learning/api/artifacts/</code>
Thayse — ML referência GlucoBench	Parcial / em andamento	<code>machine-learning/notebooks/referencia/glucobench-reference.ipynb</code>
Thayse — API	Completo (local)	<code>machine-learning/api/app/main.py</code> (<code>/health</code> , <code>/predict</code> , CORS habilitado para web)

4. Requisitos relacionados ao Bento

Os requisitos do professor Bento foram organizados com base no documento `Requisitos_Bento_GlicNutri.pdf` , referente ao Projeto GlicNutri. Esses requisitos envolvem documentação

inicial, banco de dados, autenticação, CRUD, validações, fluxo do sistema, usabilidade, auditoria, relatórios, organização do código, atualização da documentação e entrega final.

4.1 Tabela de requisitos do Bento

Nº	Requisito do Bento	O que o requisito pede	Status atual	Evidência encontrada no projeto	O que ainda precisa ser feito	Semana prevista
1	Documentação Inicial	Documento Word padronizado, definição de tecnologias e levantamento de requisitos	Completo (repo + suporte Word)	Planejamento_Final_Atividades_GlicNutri_Ajustado.md , entregas/ , WordFinalGlicNutri-ATUALIZADO.docx , entregas/WordFinalGlicNutri-ATUALIZACOES-PARA-COLAR.md	Copiar textos LGPD/resumo para o Word oficial do TCC e revisar formatação ABNT	Semana 5
2	Banco de Dados	Estrutura de tabelas, campos adequados, chaves primárias, estrangeiras e relacionamentos	Completo (repo)	supabase/migrations/ , entregas/bento/BANCO-DE-DADOS-ER-EVIDENCIAS.md , entregas/diagrama-glicnutri-a4-vertical.pdf	Print opcional do Supabase no ZIP da disciplina	Semana 5
3	Autenticação	Login de usuários, validação de credenciais e recuperação de senha	Completo (código + evidência visual)	TelaLogin.js , TelaRecuperarSenha.js , App.js ; Semana 2: evidencias-auditoria.md , login_*.png	Evidência dedicada de recuperação de senha só se o professor exigir	Semana 2
4	CRUD Completo	Cadastro, consulta, edição e exclusão lógica	Completo (repo + roteiro)	TelaCadastro.js , TelaPacientesNutricionista.js , servicoDadosPaciente.js ; entregas/bento/CRUD-VALIDACOES-ROTEIRO-EVIDENCIAS.md	Prints extra no ZIP se exigido	Semana 2
5	Validações de Dados	Validação de campos obrigatórios, CPF, CEP, email e outros dados necessários	Completo (repo + roteiro)	Formulários + servicoVerificacaoEmail.js ; roteiro CRUD	Prints de erro/sucesso no ZIP se exigido	Semana 2
6	Fluxo do Sistema	Navegação entre telas, fluxo funcional completo e persistência no banco	Completo (código + evidência parcial)	App.js , fluxos paciente/nutri/admin; Semana 2 (login + glicemia)	Vídeo demo global (pacote final)	Semana 4-5
7	Usabilidade	Interface funcional, facilidade de uso e navegação intuitiva	Completo (repo + texto)	UI RN + entregas/bento/USABILIDADE-RELATORIOS-PRINTS-TEXTO.md	Selecionar 2-3 prints finais para slides/ZIP	Semana 4-5
8	Auditoria	Registro de ações de cadastro, movimentações e logs de operações	Completo (código + evidência visual)	servicoAuditoria.js , telas admin; semana-2-auditoria/	—	Semana 4
9	Relatórios e Gráficos	Relatórios de dados, visualizações gráficas e informações gerenciais	Completo (repo + texto)	TelaHomeAdmin.js , monitoramento paciente; USABILIDADE-RELATORIOS-PRINTS-TEXTO.md	Prints de painel no ZIP/slides	Semana 4-5

Nº	Requisito do Bento	O que o requisito pede	Status atual	Evidência encontrada no projeto	O que ainda precisa ser feito	Semana prevista
10	Organização do Código	Estrutura organizada, código limpo e separação de responsabilidades	Completo (repo)	<code>src/telas/</code> , <code>src/servicos/</code> , <code>src/componentes/</code>	Descrever no Word (secção técnica)	Semana 5
11	Atualização da Documentação	Documento atualizado, corrigido conforme feedback e coerente com o sistema	Completo (repo)	Este planejamento (secção 11), <code>entregas/</code> , <code>checklists</code>	Sincronizar PDF/Word final com o repositório	Semana 5
12	Entrega Final	Sistema funcional, código em ZIP, slides e vídeo demonstrativo	Suporte completo no repo; arquivos finais pelo grupo	<code>scripts/build-zip-entrega.ps1</code> , <code>entregas/PACOTE-MONTAGEM.md</code> , <code>entregas/slides-roteiro.md</code> , <code>entregas/PACOTE-FINAL-CHECKLIST.md</code>	Correr script ZIP, gravar vídeo, exportar <code>.pptx</code>	Semana 5
13	CrITÉrios de Avaliação	Funcionamento do sistema, implementação dos requisitos, organização e qualidade da apresentação	Checklist preparado	<code>entregas/ENSAIO-FINAL-CHECKLIST.md</code> , <code>checklist-13-requisitos-bento.md</code>	Ensaio real antes da banca	Semana 5

5. Requisitos relacionados à Thayse (Machine Learning + API + dados reais)

O documento `Requisitos_Thayse_GlicNutri.pdf` descreve um sistema com pipeline de Machine Learning, incluindo pré-processamento, análise exploratória, modelagem e métricas, além de persistência e disponibilização do modelo por meio de uma API REST, utilizando FastAPI ou Flask.

5.1 Destaques obrigatórios

Para adequação completa aos requisitos, o projeto deverá:

- Refazer ou adaptar o notebook citado nos requisitos (`Diabetes_pipeline_ml.ipynb` no PDF da disciplina); **no repositório**, o pipeline principal implementado é `machine-learning/notebooks/glicnutri_ml_pipeline.ipynb`.
- Remover o uso de KaggleHub/Pima Indians Diabetes, pois se trata de dataset externo.
- Usar dados reais do GlicNutri originados do Supabase, via exportação/ETL para CSV ou leitura controlada.
- Implementar um pipeline de ML completo, com validação, EDA, treino e avaliação.
- Implementar persistência dos modelos, incluindo modelo, padronização/normalização e lista de variáveis.
- Implementar uma API FastAPI com endpoint `POST /predict`.

5.2 Tabela de requisitos da Thayse

Requisito	Status	O que precisa ser feito	Evidência ou arquivo relacionado
Arquitetura em camadas: ML Python + API REST + integração	Completo (local)	(Opcional) hospedar API para demo externa	API: <code>machine-learning/api/app/main.py</code> ; App: tela <code>src/telas/paciente/TelaPrevisaoML.js</code> + serviço <code>src/servicos/servicoMlLocal.js</code>

Requisito	Status	O que precisa ser feito	Evidência ou arquivo relacionado
Coleta/carregamento de dados em CSV + validação do dataset	Completo (local)	(Opcional) aumentar volume para métricas mais robustas	<code>machine-learning/data/glicnutri_patient_day_export.csv</code> , <code>machine-learning/data/export_manifest.json</code>
Pré-processamento: imputação, conversões e padronização	Completo (baseline)	(Opcional) ajustar por paciente e séries temporais	Notebook <code>machine-learning/notebooks/glicnutri_ml_pipeline.ipynb</code>
EDA: histogramas, boxplots, correlação e importância de variáveis	Completo (baseline)	(Opcional) expandir plots e narrativa no documento final	Notebook executado: <code>machine-learning/notebooks/glicnutri_ml_pipeline.executed.ipynb</code>
Treinamento e avaliação com métricas adequadas	Completo (baseline)	(Opcional) calibrar thresholds e métricas por classe	Notebook + <code>machine-learning/api/artifacts/training_meta.json</code>
Cobertura de 4 problemas: classificação, regressão, clusterização e similaridade/recomendação	Completo	(Opcional) ajustar targets clínicos	<code>machine-learning/notebooks/glicnutri_ml_pipeline.ipynb</code>
Persistência: modelo, padronização e lista de variáveis	Completo	(Opcional) versionar artefatos por data/seed	<code>machine-learning/api/artifacts/*.joblib</code> , <code>training_meta.json</code>
API REST com endpoint <code>POST /predict</code>	Completo (local + web)	(Opcional) hospedar no Render	<code>machine-learning/api/app/main.py</code> com CORS habilitado; <code>uvicorn app.main:app</code>

5.3 Definição oficial do dataset real (para o grupo não divergir)

Esta secção fixa um único “contrato” de dataset para o projeto, alinhado ao export existente (`machine-learning/scripts/export_supabase_csv.py`) e ao pipeline (`machine-learning/notebooks/glicnutri_ml_pipeline.ipynb`).

5.3.1 Unidade (granularidade)

- Uma linha = 1 paciente em 1 dia (chave: `patient_id` + `dia`).
- Fonte: export Supabase → CSV “paciente-dia”.

5.3.2 Janela temporal (regra padrão)

- Export “rolling window” padrão: últimos 60 dias (`--days 60`).
- Para treinos mais robustos, pode-se aumentar para 90–365 dias mantendo a mesma estrutura.
- Evidência reprodutível obrigatória: `machine-learning/data/export_manifest.json` (hash + contagem + colunas + janela).

5.3.3 Colunas oficiais do CSV (schema)

O dataset oficial deve conter (nomenclatura exata do CSV):

- Identificação e tempo:

- `patient_id` (uuid em texto)
- `dia` (YYYY-MM-DD)
- Glicemia agregada no dia:
- `glucose_mean_mg_dl`, `glucose_min_mg_dl`, `glucose_max_mg_dl`
- `n_leituras_glicemia`
- Refeições agregadas no dia:
- `carbs_sum_g`, `kcal_sum`, `protein_sum_g`, `fat_sum_g`
- `n_refeicoes_ia`
- Medicação agregada no dia:
- `n_eventos_medicao`

5.3.4 Features oficiais (entrada do modelo / API)

As features usadas no treino e no endpoint `/predict` são:

- `n_leituras_glicemia`
- `carbs_sum_g`
- `kcal_sum`
- `protein_sum_g`
- `fat_sum_g`
- `n_refeicoes_ia`
- `n_eventos_medicao`

Observação: os campos de glicemia (`glucose_*`) são usados como **alvo** e para EDA; não entram como feature no `/predict` (para evitar vazamento do alvo).

5.3.5 Alvos oficiais (o que o modelo aprende a prever)

O projeto cobre 4 tarefas (requisito da disciplina), com estes alvos/definições:

- **Classificação (alvo):** `target_glucose_high = 1` se `glucose_mean_mg_dl >= 150`, senão `0`.
- **Regressão (alvo):** prever `glucose_mean_mg_dl`.
- **Clusterização (não supervisionado):** KMeans no espaço de features.
- **Similaridade (não supervisionado):** NearestNeighbors no espaço de features.

5.3.6 Regras de qualidade e limpeza (obrigatórias)

- **Chave única:** não pode haver duplicado de (`patient_id`, `dia`). Se houver, o export deve ser corrigido ou agregações revisadas.
- **Nulos nas features:** preencher com `0` (`fillna(0)`) para:
- `n_leituras_glicemia`, `carbs_sum_g`, `kcal_sum`, `protein_sum_g`, `fat_sum_g`, `n_refeicoes_ia`, `n_eventos_medicao`.
- **Nulos no alvo:**
- Linhas com `glucose_mean_mg_dl` nulo **não entram** nas tarefas supervisionadas (classificação e regressão).
- Essas linhas podem permanecer no CSV para auditoria/EDA, mas são removidas no treino supervisionado.
- **Filtro de pacientes:** somente pacientes com `paciente.excluido = false` (já aplicado no SQL do export).

5.3.7 Evidências mínimas que o grupo deve guardar (para entrega)

- `machine-learning/data/export_manifest.json` (prova reprodutível do export)
- `machine-learning/data/glicnutri_patient_day_export.csv` (anexo no ZIP se exigido)
- `machine-learning/notebooks/glicnutri_ml_pipeline.executed.ipynb` (ou prints das saídas principais)
- `machine-learning/api/artifacts/` (artefatos joblib + `training_meta.json`)
- Evidência de chamada `/predict` (print do app na tela "Previsão (IA)" ou teste em PowerShell/Python)

Resumo do modelo aplicado no GlicNutri (explicação simples e completa)

O GlicNutri usa um dataset **paciente-dia** (uma linha por paciente por dia) para transformar registros do dia (glicemia, refeições e medicação) em previsões e padrões úteis. O pipeline de Machine Learning foi implementado no notebook `machine-learning/notebooks/glicnutri_ml_pipeline.ipynb` e os modelos treinados são persistidos em `machine-learning/api/artifacts/` para serem usados pela API (`POST /predict`).

O que entra no modelo (features):

- Quantidade de leituras de glicemia no dia (`n_leituras_glicemia`)
- Totais nutricionais do dia (`carbs_sum_g` , `kcal_sum` , `protein_sum_g` , `fat_sum_g`)
- Quantidade de refeições registradas (`n_refeicoes_ia`)
- Quantidade de eventos de medicação (`n_eventos_medicao`)

O que o modelo resolve (4 problemas exigidos pela disciplina):

1) Classificação (supervisionado)

- **Pergunta:** "Com base nos hábitos/resumos do dia, este dia tende a ter glicemia média elevada?"
- **Alvo:** `target_glucose_high = 1` quando `glucose_mean_mg_dl >= 150` (senão 0).
- **Saída na API:** `prob_glucose_elevada` (probabilidade) e `classe_glucose_elevada` (0/1).

2) Regressão (supervisionado)

- **Pergunta:** "Qual a glicemia média estimada (mg/dL) para um dia com essas características?"
- **Alvo:** `glucose_mean_mg_dl` .
- **Saída na API:** `glucose_mean_previsto_mg_dl` .

3) Clusterização (não supervisionado)

- **Pergunta:** "Quais perfis de dias existem no comportamento alimentar/medicação/leituras?"
- **Técnica:** KMeans sobre o mesmo conjunto de features (sem usar o alvo).
- **Saída na API:** `cluster_id` (identificador do grupo).

4) Similaridade (não supervisionado)

- **Pergunta:** "Quais dias do histórico são mais parecidos com o dia atual?"
- **Técnica:** NearestNeighbors no espaço de features.
- **Saída na API:** `vizinhos_mais_proximos_indices` (índices dos dias mais próximos na base).

Regras importantes do pipeline (para evitar erro e manter coerência):

- Linhas com `glucose_mean_mg_dl` **nulo** não entram no treino supervisionado (classificação/regressão), pois o alvo é desconhecido.
- Nulos nas features são preenchidos com **0**, garantindo que o modelo rode mesmo em dias sem refeição/medicação registrada.
- O export já filtra pacientes com `excluido = false` .

Como o paciente vê isso no app (teste local):

- Foi criada uma tela de teste "**Previsão (Machine Learning)**" (`PacientePrevisaoML`) onde o usuário preenche os 7 campos e chama o endpoint `/predict` , visualizando o resultado (probabilidade de glicemia elevada, classe, glicemia média prevista e cluster). Em ambiente web, a API permite CORS para esse teste local.

6. Planejamento semanal até 31/05/2026

O cronograma abaixo organiza as entregas semanais necessárias para finalizar o projeto até 31/05/2026.

Semana	Período	Atividades	Responsável	Tempo estimado	Entrega da semana	Reunião semanal
Semana 1	28/04 a 03/05	Consolidar os requisitos do Bento e da Thayse; mapear o que já existe no app e no Supabase; definir o dataset real de ML; relacionar os 13 requisitos do Bento com evidências iniciais	Integrante 1 + Integrante 2	12 a 16h	Checklist de requisitos + especificação do dataset ML	03/05, 30 a 45 min
Semana 2	04/05 a 10/05	Implementar estratégia reprodutível Supabase para CSV; adaptar notebook para ler CSV real; executar validações e EDA inicial; testar autenticação, CRUD, validações e fluxo principal do sistema	Integrante 2 + Integrante 4	14 a 18h	CSV reprodutível + notebook com validação/EDA + evidências dos fluxos principais — índice no Git: entregas/README-ENTREGA-SEMANA-2.md	10/05, 30 a 45 min
Semana 3	11/05 a 17/05	Treinar 4 pipelines com dados reais: classificação, regressão, clusterização e similaridade; gerar métricas; salvar figuras; selecionar versão do modelo	Integrante 2 + Integrante 3	16 a 22h	Resultados, métricas, figuras e decisão do modelo final	17/05, 45 a 60 min
Semana 4	18/05 a 24/05	Criar backend Python organizado; implementar persistência; criar API FastAPI <code>POST /predict</code> ; testar localmente; consolidar auditoria, logs, usabilidade, relatórios e gráficos	Integrante 3 + Integrante 4	16 a 20h	API funcional + contrato JSON + evidências de auditoria, logs, relatórios e gráficos	24/05, 30 a 45 min
Semana 5	25/05 a 31/05	Fazer integração mínima app com API ou simulação controlada; finalizar documentação do Bento e da Thayse; preparar ZIP, slides, vídeo, evidências e ensaio final	Todos	18 a 24h	Pacote final: documento, slides, vídeo, ZIP, evidências e checklist final	31/05, 60 min

Situação real — Semana 2 (maio/2026): **COMPLETA** no repositório — parte **Bento / auditoria** com evidência visual validada ([entregas/bento/semana-2-auditoria/](#)) e parte **Thayse (CSV/ML/API)** com export real + manifest + notebook executado + artefatos + API + integração no app (ver [entregas/thayse/RESUMO-ENTREGA-ML.md](#)).

7. Divisão de tarefas entre o grupo

Para equilibrar responsabilidades e garantir rastreabilidade, a divisão inicial será:

Integrante	Foco principal	Tarefas principais
Integrante 1	Documentação e requisitos do Bento	Documento final acadêmico, slides, rastreabilidade requisito-evidência, texto de LGPD e roteiro de apresentação
Integrante 2	Dados e notebook de ML	Dataset real Supabase para CSV, refatoração do notebook, EDA, treinamento e métricas dos 4 pipelines
Integrante 3	Backend Python e API	Estrutura Python organizada, persistência com joblib, FastAPI <code>POST /predict</code> , testes e documentação de execução
Integrante 4	Evidências do app e qualidade	Evidências das telas e fluxos, CRUD, exclusão lógica, movimentações, auditoria, logs, relatórios e checklist final
Todos	Validação final	Participar das reuniões semanais, revisar entregas, testar o sistema e ensaiar apresentação

8. Reuniões semanais

O grupo realizará reuniões semanais de acompanhamento para garantir o progresso contínuo e a validação do que será apresentado ao professor. Em cada reunião:

- Cada integrante deverá apresentar sua parte no próprio computador, mostrando evidências e resultados, como telas, relatórios, notebook, API e documentos.
- Pelo menos um integrante deverá estar presente semanalmente para apresentar ao professor o status da semana e demonstrar o que foi produzido.
- Ao final de cada reunião, o grupo deverá atualizar o checklist de acompanhamento e registrar as pendências para a próxima semana.

9. Riscos e cuidados

Risco/Cuidado	Possível impacto	Ação de mitigação
Falta de conferência final dos requisitos do Bento	Algum requisito pode ficar sem evidência no documento ou na apresentação	Usar a tabela dos 13 requisitos do Bento como checklist oficial durante as semanas de execução
Dados reais insuficientes no Supabase	Modelos de ML pouco confiáveis ou inviáveis	Definir dataset por agregação, como paciente-dia ou janelas de tempo, e preparar base mínima para demonstração
Dificuldade de adaptar o notebook	Atraso no cronograma da Thayse	Refatorar por etapas: leitura CSV, validação, EDA, modelos e persistência
Integração API com aplicativo	Atrasos técnicos na fase final	Priorizar API funcional primeiro; se necessário, realizar integração mínima ou simulação controlada na Semana 5
Atrasos do grupo	Comprometimento da entrega final	Manter reuniões semanais, divisão clara de tarefas, entregas pequenas por semana e checklist de verificação

10. Conclusão

Este documento organiza as atividades do Projeto GlicNutri por semana, com divisão de responsabilidades e entregas verificáveis, visando garantir a conclusão até 31 de maio de 2026. O planejamento contempla os requisitos do professor Bento, relacionados à evolução do sistema, banco de dados, autenticação, CRUD, validações, auditoria, relatórios, documentação e entrega final, além dos requisitos da professora Thayse, relacionados à entrega de Machine Learning com dados reais do Supabase, persistência e API de predição.

Com a execução deste plano, o grupo terá maior controle sobre as pendências, reduzirá riscos de atraso e conseguirá apresentar evidências claras de funcionamento, implementação dos requisitos, organização do projeto e qualidade da apresentação final.

11. Checklist final do grupo

Item	Status
Índice e manual Semana 2 no repositório (entregas/README-ENTREGA-SEMANA-2.md , SEMANA-2-PASSO-A-PASSO.md , atalhos export)	[x]
Conferir os 13 requisitos do Bento e relacionar cada um com uma evidência do sistema ou da documentação	[x] entregas/bento/checklist-13-requisitos-bento.md

Item	Status
Consolidar checklist da Thayse e rastrear cada item para um entregável	[x] <code>entregas/thayse/RESUMO-ENTREGA-ML.md</code>
Definir dataset real do GlicNutri: variáveis, objetivo e regras	[x] seção 5.3 (dataset oficial paciente-dia)
Implementar exportação/ETL Supabase para CSV reprodutível	[x] script <code>machine-learning/scripts/export_supabase_csv.py</code>
Remover/isolar KaggleHub/Pima no notebook legado	[x] referência GlicNutri : <code>machine-learning/notebooks/glicnutri_ml_pipeline.ipynb</code> (paciente-dia). Referência GlucoBench : <code>machine-learning/notebooks/referencia/glucobench-reference.ipynb</code> . Pima/Kaggle removidos do fluxo do projeto
Executar validação do dataset real: tipos, nulos e integridade	[x] export + manifest (<code>machine-learning/data/export_manifest.json</code>) + validação no notebook
Refazer EDA com dados reais: histogramas, boxplots, heatmap e importância de variáveis	[x] baseline no notebook executado (<code>machine-learning/notebooks/glicnutri_ml_pipeline.executed.ipynb</code>)
Treinar e avaliar os 4 pipelines com dados reais	[x] classificação/regressão/cluster/similaridade executados + artefatos gerados
Persistir artefatos: modelo, padronização e variáveis	[x] <code>machine-learning/api/artifacts/*.joblib</code> , <code>training_meta.json</code>
Criar backend Python organizado e reproduzível	[x] pasta <code>machine-learning/api/</code> + <code>requirements.txt</code>
Implementar API FastAPI com <code>POST /predict</code> e testar com exemplos	[x] <code>machine-learning/api/app/main.py</code> + CORS (web)
Integração do app com a API local de ML (tela paciente)	[x] <code>src/telas/paciente/TelaPrevisaoML.js</code> , <code>src/servicos/servicoMlLocal.js</code> , rota <code>PacientePrevisaoML</code> e menu paciente
Consolidar evidências do app: login, CRUD, exclusão lógica e processos	[x] Semana 2 Bento: <code>entregas/bento/semana-2-auditoria/</code> com PNG em <code>prints/</code> e docs alinhados
Consolidar auditoria/logs no repositório (evidências Semana 2)	[x] <code>evidencias-auditoria.md</code> , <code>checklist-auditoria.md</code> , <code>STATUS-SEMANA-2.md</code> ; Evidência visual validada com capturas reais no repositório
Roteiros/texto prontos para fechar parciais do Bento (BD/CRUD/UX/relatórios)	[x] <code>entregas/bento/BANCO-DE-DADOS-ER-EVIDENCIAS.md</code> , <code>entregas/bento/CRUD-VALIDACOES-ROTEIRO-EVIDENCIAS.md</code> , <code>entregas/bento/USABILIDADE-RELATORIOS-PRINTS-TEXTO.md</code>
Consolidar texto LGPD no documento Word final	[x] texto em <code>entregas/LGPD-TEXTO-CURTO.md</code> + suplemento <code>WordFinalGlicNutri-ATUALIZADO.docx</code> / <code>entregas/WordFinalGlicNutri-ATUALIZACOES-PARA-COLAR.md</code> — copiar para o Word oficial do TCC
Consolidar relatórios e gráficos para apresentação	[x] <code>entregas/bento/USABILIDADE-RELATORIOS-PRINTS-TEXTO.md</code> — capturas finais no ZIP se a disciplina exigir
Preparar slides e roteiro de demonstração	[x] roteiro <code>entregas/slides-roteiro.md</code> — ficheiro .pptx é manual
Preparar ZIP do projeto e vídeo demonstrativo	[x] suporte <code>scripts/build-zip-entrega.ps1</code> + <code>entregas/PACOTE-MONTAGEM.md</code> — correr script, anexar ZIP e gravar vídeo manualmente
Ensaar apresentação final e revisar checklist de entrega	[x] <code>entregas/ENSAIO-FINAL-CHECKLIST.md</code> + <code>entregas/PACOTE-FINAL-CHECKLIST.md</code> — executar ensaio na vossa máquina

11.1 O que só o grupo executa (fora do Git)

- Exportar e subir o `.pptx` e o vídeo finais.
- Correr `build-zip-entrega.ps1` e juntar ao pacote **CSV real** / prints extra, se obrigatório.
- **Ensaio real** (uvicorn + `npm run start` + Previsão IA) na semana da entrega.

12. Próximos passos imediatos do grupo

1. **Pacote e ensaio:** seguir `entregas/PACOTE-MONTAGEM.md` , `entregas/ENSAIO-FINAL-CHECKLIST.md` e `entregas/PACOTE-FINAL-CHECKLIST.md` (ZIP, vídeo, `.pptx` , ensaio local).
2. **Semana 2 — Thayse (fechado localmente):** export executado (CSV + `export_manifest.json`), notebook executado (EDA + treino + artefatos), API local rodando e integrada ao app (tela paciente “Previsão (IA)”). Próximo passo é **capturar evidências** (prints e métricas) para o pacote acadêmico e, se permitido, commitar `machine-learning/data/export_manifest.json` .
3. **Reunião 10/05:** seguir `entregas/demo-reuniao-semana-2.md` ; declarar plano B (sample + limitações) se o volume real ainda for insuficiente.
4. **Checklist Bento (13):** mantido em `entregas/bento/checklist-13-requisitos-bento.md` com evidências no repositório; atualizar apenas se o professor pedir provas adicionais.
5. (Concluído) Dataset oficial do ML definido na secção 5.3 (paciente-dia + features + alvos + regras).
6. **Semana 3:** reforçar métricas e narrativa (opcional: mais dados, melhor validação, gráficos e explicação clínica) — o baseline já está funcional.
7. Estabelecer rotina de reunião semanal e uma entrega mínima por semana, com responsáveis e validação coletiva.